

PRIORITY HOME MONITOR

Digital Platform

Technical Architecture & Implementation Plan

Revision 5 | May 2026

What's New in Revision 5

Revision 5 expands the platform from a pure lead-generation hub into a **hybrid lead-generation and e-commerce platform**. All Revision 4 content is preserved. The following architectural additions are integrated throughout every relevant section:

- **E-commerce product catalog** with a public-facing /shop storefront
- **Stripe payment integration** — the single authorized exception to the AWS-native rule
- **S3 presigned URL pipeline** for direct admin image uploads
- **Extended PostgreSQL schema**: products table + new ENUMs + No Hard Delete policy
- **Stripe webhook handler** at /api/webhooks/stripe
- **Admin Product Catalog CRUD** dashboard with RBAC clarification
- **Cost model updated**: Stripe adds \$0.00 fixed monthly cost

Table of Contents

1.	Executive Summary	3
2.	Frontend Architecture	5
3.	Backend Architecture	8
4.	Database Design	10
5.	AWS Infrastructure	14
6.	RAG Chatbot Implementation	19
7.	Admin Panel	23
8.	Analytics — Google Analytics 4	26
9.	Security & HIPAA-Ready Structure	28
10		
.	Cost Optimization	31
11		
.	Scalability Architecture	33
12		
.	Implementation Roadmap	35
13		
.	Open Questions for Stakeholder Sign-Off	45

1. Executive Summary

Priority Home Monitor requires a professional-grade web platform serving three distinct purposes: an information hub for patients and families, a lead-generation and referral engine for healthcare providers, and — new in Revision 5 — a **public-facing e-commerce storefront** for medical monitoring devices. The platform must handle medical-adjacent content responsibly while driving conversions through a frictionless intake flow and enabling direct product transactions.

1.1 Core Deliverables

- Multi-page, SEO-optimized Next.js 14 frontend covering all 16 program routes plus the new /shop and /admin/products routes from the PHM site structure.
- Backend split: Next.js API routes for lightweight operations, AWS Lambda for AI processing and PDF ingestion.
- PostgreSQL with pgvector on Amazon RDS for lead management, vector storage, and product catalog in a single instance.
- Fully AWS-native AI stack: Bedrock Titan Text Embeddings v2 for embeddings, Claude Haiku for generation. No external AI API dependencies.
- RAG chatbot restricted to verified PHM content. Guide-only. Never collects data. Always redirects to /refer.
- Admin dashboard protected by Amazon Cognito: staff can view and manage all form submissions and the product catalog with RBAC.
- E-commerce product catalog** with a public /shop page and a protected /admin/products inventory management interface.
- Stripe payment integration** for transactional checkout, with a webhook handler at /api/webhooks/stripe to write confirmed orders to PostgreSQL.
- Google Analytics 4 with full event and conversion tracking on all CTAs, form submissions, and shop transactions.
- HIPAA-ready data architecture that can be upgraded to full HIPAA compliance without rebuilding the platform.

1.2 Architecture Principles

Principle	Implementation
AWS-native first (Stripe exception)	Bedrock for AI, SES for email, Cognito for auth, CloudFront for CDN. Stripe is the single authorized
Serverless-first	Lambda for AI and ingestion. Next.js API routes for forms and product CRUD. No always-on serve
Least-privilege IAM	Every Lambda has its own role scoped to exactly what it needs. Amplify has zero direct database
HIPAA-ready	No PHI collected today. Schema and VPC structured so a BAA upgrade requires no rebuild.
Local-first development	The entire website and chatbot are built and validated locally before a single AWS deployment. Th
Beginner-friendly	Step-by-step roadmap in Section 12. Every AWS action is a specific console or CLI command. No
Scale to thousands	CloudFront CDN for static pages, Lambda concurrency controls, HNSW vector search, RDS read

1.3 Build Timeline

Phase	Weeks	Focus
Phase 1	1	Local setup: Next.js, design system, AWS prerequisites (RDS + Cognito provisioned but used local)
Phase 2	1–3	Build the full website locally: all 16 pages + /shop storefront, forms, admin panel + /admin/products
Phase 3	3–4	Build and test AI chatbot locally: PDF ingestion, RAG Lambda, chatbot widget
Phase 4	4–5	Deploy everything to AWS: Amplify CI/CD, CloudFront, WAF, Route 53, production config, Stripe I
Phase 5	5–6	QA, cross-browser testing, Lighthouse, stakeholder review, production go-live

2. Frontend Architecture

2.1 Tech Stack

Layer	Technology	Rationale
Framework	Next.js 14 (App Router)	SSR/SSG, file-based routing, native API routes. App Router required for str
Hosting	AWS Amplify Gen 2	Gen 2 is the only Amplify generation that supports Next.js 14 App Router. C
Styling	Tailwind CSS	Utility-first. Consistent clinical aesthetic. No CSS sprawl across large teams
Components	Shadcn/UI	Accessible, pre-built components: Cards, Buttons, Dialogs, Form elements.
Icons	Lucide React	Lightweight, consistent icon set. No license issues.
Forms	React Hook Form + Zod	Type-safe form validation with minimal re-renders. Zod schemas shared wi
State	Zustand	Lightweight global state for chatbot session context (last 5 turns) and cart s
HTTP	Native fetch + Axios	Server Components use native fetch. Axios for client-side API calls.
Image Preview	URL.createObjectURL	Admin product image uploads use URL.createObjectURL for local in-brows
Auth (Admin)	AWS Amplify Auth SDK	Wraps Cognito. Handles login, token refresh, and logout with minimal boiler
Payments	Stripe.js + @stripe/react-stripe-js	Client-side Elements library for PCI-compliant card collection. Card data ne
Analytics	Google Analytics 4	Event and conversion tracking. Installed via Next.js Script component.

2.2 All Page Routes

Route	Page Title	Primary CTA
/	Home: Hero, Benefits, Team, Programs overview	Refer a Patient
/programs	All Programs overview	View All Programs
/programs/copd	COPD — Respiratory Care	Start COPD Monitoring
/programs/hypertension	Hypertension Monitoring	Enroll in Hypertension Care
/programs/ccm/diabetes	Diabetes — Chronic Care Management	Explore Diabetes Care
/programs/ccm/heart-failure	Heart Failure — CCM	Start Heart Failure Program
/programs/osa	Obstructive Sleep Apnea	Explore OSA Program
/programs/sleep/pediatric	Pediatric Sleep Monitoring	Explore Pediatric Sleep
/programs/sleep/adult	Adult Sleep Monitoring	Start Adult Sleep Program
/programs/sleep/ent	ENT Sleep Program	Consult ENT Sleep Program
/programs/wellness	Wellness: Mental, Nutrition, Weight	Explore Mental Wellness

Route	Page Title	Primary CTA
/programs/nutrition/diet/meals	Prepared Meal Program (CookUnity)	Visit CookUnity
/programs/fall-detection	Fall Detection — Safety & Monitoring	Deploy Fall Detection
/contact	Contact Us	Request a Consultation
/refer	Refer a Patient — intake form	Submit Referral
/platform	Platform Demo Request	Request a Platform Demo
/shop	E-Commerce Storefront — Medical Monitoring Devices	Get Device / Buy Now
/admin/products	Product Catalog Management (Protected)	Add New Product

New in Revision 5: /shop and /admin/products are highlighted above. /shop is publicly accessible. /admin/products requires a valid Cognito JWT with admin or superadmin role.

2.3 UI/UX Design System

The target audience is older adults, patients with chronic conditions, and healthcare providers under time pressure. The design must be minimal, modern, and visually impressive while being effortless to use. Every element earns its place. Zero clutter.

Typography

Element	Specification	Reason
Body font	Inter or system-ui, minimum 16px	Legibility for older users. Never below 16px. Full stop.
Heading scale	H1: 40–48px H2: 28–32px H3: 22–28px H4: 18–20px	Spanned 18–20px. Patient lands on page, sees the key message.
Line height	1.6 for body, 1.2 for headings	Reduces eye fatigue. Critical for medical information.
Letter spacing	0.01em body, -0.02em for large display headings	Modern, clinical feel without sacrificing legibility.
Section padding	Minimum 80px vertical per section	Breathing room. Healthcare content needs space to not feel over-
Content max-width	1100px centered	Prevents uncomfortable eye travel on wide monitors.
Font weight	400 for body, 600 for subheadings, 700 for headings	Clear hierarchy and readability. Avoid 300 (too light) and 900 (too aggressive).

Color System

Token	Hex Value	Usage
--color-primary	#1B3A5C (Navy)	Main headings, navbar, footer, primary CTA borders
--color-accent	#0D7377 (Teal)	Primary CTA fill, active states, links, highlights
--color-surface	#F8FAFC (Off-white)	Page backgrounds, card fills. Not pure white — reduces glare.
--color-border	#E2EBF4	Card, input, and divider borders. Subtle but visible.

Token	Hex Value	Usage
--color-text-primary	#1A1A2E	All body text. Contrast ratio 21:1 on surface. Passes WCAG AAA.
--color-text-muted	#5A6A7A	Captions, secondary info, helper text. 4.7:1 on surface — passes AA.
--color-success	#1A6B4A	Enrolled status, form confirmation, success badges, order confirmed
--color-warning	#C75B00	Pending review, processing states, order pending
--color-danger	#9B1C1C	Form errors, emergency fallback text, validation failures
--color-stripe	#635BFF	Stripe-related UI elements only (payment badge, secure checkout label)

3. Backend Architecture

3.1 Split Architecture: Next.js API Routes + AWS Lambda

The backend is split across two layers by workload type. Lightweight synchronous operations — form submissions, chatbot message routing, input validation, product CRUD, and payment webhook handling — are handled by Next.js API routes deployed on AWS Amplify Gen 2. These run at the edge and have near-zero cold-start overhead.

Heavy async operations — PDF ingestion, embedding generation via Bedrock Titan, email dispatch — run as AWS Lambda functions. Lambda is triggered by S3 events for ingestion and invoked by the API proxy for RAG. This keeps the frontend deployment lean while isolating expensive AI processing into independently scalable serverless functions with their own IAM roles.

3.2 API Endpoints

Endpoint	Method	Handler	Description
/api/refer	POST	Next.js API Route	Accepts referral form. Validates with Zod. Writes to RDS
/api/consult	POST	Next.js API Route	Consultation intake. Same pipeline as /api/refer.
/api/contact	POST	Next.js API Route	General contact form. Logs to DB. Triggers SES notificat
/api/chat	POST	Next.js → Lambda proxy	Sanitizes input. Invokes RAG Lambda. Streams Bedrock
/api/admin/leads	GET	Next.js API Route	Paginated leads list. Cognito JWT required. Returns filter
/api/admin/leads/:id	PATCH	Next.js API Route	Updates lead status. Cognito JWT + admin role required.
/api/admin/products	GET/POST/PATCH	Next.js API Route	Protected CRUD endpoints for managing the product cat
/api/upload/presigned	POST	Next.js API Route	Generates an Amazon S3 presigned PUT URL, enabling
/api/webhooks/stripe	POST	Next.js API Route	Intercepts and validates Stripe payment events (charge.s
/ingest (internal)	—	Lambda (S3 trigger)	Fires on new PDF in S3 knowledge-base bucket. Chunks

New in Revision 5: /api/admin/products, /api/upload/presigned, and /api/webhooks/stripe (highlighted above) are the three new endpoints supporting the e-commerce layer.

3.3 Request Validation

Every POST endpoint validates the request body with a Zod schema before touching the database or any AI service. Invalid requests return HTTP 400 with a structured error object. The Stripe webhook endpoint is the sole exception: it must receive the raw unparsed body for signature verification and therefore bypasses the standard Zod middleware.

Field	Validation Rule
Patient / provider name	string, 2–100 characters, stripped of HTML and special characters
Phone	E.164 format validation and coercion on input blur

Field	Validation Rule
Email	RFC 5321 compliant format check
Chat message	Max 500 characters, all HTML tags stripped, validated against an allow-list pattern before reach
Cognito JWT (admin routes)	Verified against Cognito JWKS on every request. Expired tokens return 401. Tampered tokens
reCAPTCHA token	Sent with all form submissions. Verified server-side against Google reCAPTCHA API before pr
Stripe webhook signature	Verified using stripe.webhooks.constructEvent() with the STRIPE_WEBHOOK_SECRET envir
Product fields (admin)	name: string 2–120 chars. price: positive NUMERIC. type: enum check. stock_status: enum ch

3.4 Email Notifications (AWS SES)

Every referral and consultation triggers two SES emails: a confirmation to the submitter and an internal notification to the PHM team inbox. SES is preferred over third-party SMTP (SendGrid, Mailgun) for native AWS integration, consistently high deliverability to US healthcare email domains, and zero external vendor dependency.

Pitfall: SES starts in sandbox mode. It can only email verified addresses until you request production access. Do this early — approval takes 24–48 hours. See Step 9 in Section 12.

4. Database Design

4.1 Why PostgreSQL with pgvector

The project requires two storage types: relational data (leads, admin accounts, product catalog, order records) and vector embeddings (AI knowledge base). PostgreSQL with pgvector on a single Amazon RDS instance handles all of these. This eliminates the cost and operational overhead of a separate vector database such as Pinecone (~\$70/month) or Weaviate while providing full SQL capabilities for lead management and the e-commerce layer.

4.2 ENUM Types

Revision 5 adds two new ENUM types to support the product catalog. These are defined once in the database and shared across tables.

ENUM Name	Values	Usage
lead_type	'referral', 'consultation', 'contact'	Which form was submitted (unchanged from Revision 4)
lead_status	'new', 'contacted', 'enrolled', 'closed'	Lead lifecycle state (unchanged from Revision 4)
admin_role	'admin', 'superadmin'	Controls dashboard RBAC (unchanged from Revision 4)
product_stock_status	'in_stock', 'out_of_stock', 'archived'	Controls product visibility. Archived items are never shown p
product_type	'individual', 'bundle'	Distinguishes single devices from bundled monitoring kits.

4.3 Schema

Table 1: leads

Stores all form submissions from the public site. This is the primary business data table. Unchanged from Revision 4.

Column	Type	Notes
id	UUID PRIMARY KEY	Generated via gen_random_uuid()
type	ENUM('referral', 'consultation', 'contact')	Which form was submitted
patient_name	VARCHAR(100)	
provider_name	VARCHAR(100)	Referring provider. Nullable for contact forms.
phone	VARCHAR(20)	Stored as-is after E.164 validation at the API layer
email	VARCHAR(255)	
condition_interest	VARCHAR(100)	e.g. 'COPD', 'Diabetes', 'Fall Detection'
message	TEXT	Free-form notes. No clinical data should be entered here.
source_page	VARCHAR(255)	URL path of the page where the form was submitted

Column	Type	Notes
created_at	TIMESTAMPTZ	DEFAULT NOW()
status	ENUM('new', 'contacted', 'enrolled', 'closed')	Default: new. Updated by admin.

Table 2: products (New in Revision 5)

Stores the medical monitoring device catalog. All products are managed exclusively through /admin/products. The No Hard Delete policy applies: product records are never removed via DELETE statements. Discontinued or retired items are masked by setting stock_status to 'archived', preserving full historical audit trails for past orders.

Column	Type	Notes
id	UUID PRIMARY KEY	Generated via gen_random_uuid()
name	VARCHAR(255) NOT NULL	Display name shown on /shop. Maximum 255 characters.
description	TEXT	Full product description. Supports plain text. Rendered as paragraphs.
price	NUMERIC(10, 2) NOT NULL	Stored as exact decimal. Never use FLOAT for currency. Price in USD.
type	product_type NOT NULL	Either 'individual' (single device) or 'bundle' (monitoring kit).
stock_status	product_stock_status NOT NULL	Default: 'in_stock'. Set to 'archived' to hide from /shop without deleting.
image_url	VARCHAR(512)	Full HTTPS URL of the product image stored in S3. Populated by product image service.
created_at	TIMESTAMPTZ NOT NULL	DEFAULT NOW()
updated_at	TIMESTAMPTZ NOT NULL	DEFAULT NOW(). Should be updated via trigger or explicit SET clause.

No Hard Delete Policy: The products table enforces a strict no-deletion rule. Inventory is never removed using DELETE SQL statements. Instead, items are retired by setting stock_status = 'archived'. This preserves the integrity of historical order records that reference a product_id, prevents foreign key violations, and maintains a full audit trail for compliance purposes. The /shop page filters WHERE stock_status != 'archived'. Admin views show all records including archived items.

Table 3: knowledge_chunks

The RAG vector store. Unchanged from Revision 4.

Column	Type	Notes
id	UUID PRIMARY KEY	
source_doc	VARCHAR(255)	Filename of the source PDF
chunk_index	INTEGER	Order of the chunk within the document
content	TEXT	Raw text chunk, 300–500 tokens
embedding	VECTOR(1024)	Bedrock Titan Text Embeddings v2 output. 1024 dimensions.
cluster_tag	VARCHAR(2)	A through E. Used for centroid-based intent pre-filtering.
created_at	TIMESTAMPTZ	DEFAULT NOW()

Table 4: cluster_centroids

Stores the mean embedding vector for each of the five topic clusters. Unchanged from Revision 4.

Table 5: admin_users

Local reference table for admin accounts. Cognito manages authentication. Unchanged from Revision 4.

4.4 SQL Setup Script (Updated for Revision 5)

Run this against your RDS instance after provisioning. Includes all Revision 4 tables plus the new products table and ENUMs.

```
-- Run this against your RDS instance after provisioning
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Revision 4 ENUMs
CREATE TYPE lead_type AS ENUM ('referral', 'consultation', 'contact');
CREATE TYPE lead_status AS ENUM ('new', 'contacted', 'enrolled', 'closed');
CREATE TYPE admin_role AS ENUM ('admin', 'superadmin');

-- Revision 5 ENUMs (new)
CREATE TYPE product_stock_status AS ENUM ('in_stock', 'out_of_stock', 'archived');
CREATE TYPE product_type AS ENUM ('individual', 'bundle');

-- leads (unchanged)
CREATE TABLE leads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type lead_type NOT NULL,
  patient_name VARCHAR(100) NOT NULL,
  provider_name VARCHAR(100),
  phone VARCHAR(20),
  email VARCHAR(255) NOT NULL,
  condition_interest VARCHAR(100),
  message TEXT,
  source_page VARCHAR(255),
  status lead_status NOT NULL DEFAULT 'new',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- products (new in Revision 5)
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
```

```
    type product_type NOT NULL,  
    stock_status product_stock_status NOT NULL DEFAULT 'in_stock',  
    image_url VARCHAR(512),  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- knowledge_chunks (unchanged)  
CREATE TABLE knowledge_chunks (...); -- see Revision 4 schema  
  
-- cluster_centroids (unchanged)  
CREATE TABLE cluster_centroids (...); -- see Revision 4 schema  
  
-- admin_users (unchanged)  
CREATE TABLE admin_users (...); -- see Revision 4 schema  
  
-- HNSW index (run after knowledge_chunks is populated)  
-- CREATE INDEX ON knowledge_chunks  
--     USING hnsw (embedding vector_cosine_ops)  
--     WITH (m = 16, ef_construction = 64);
```

5. AWS Infrastructure

5.1 Service Map

Component	AWS Service / Provider	Purpose
Frontend Hosting	AWS Amplify Gen 2	SSL, CDN, CI/CD from GitHub. Gen 2 required for Next.js 14 App Router.
Static Assets, PDFs & Product Images	AWS S3	Images, fonts, PHM knowledge-base PDFs, and — new in Revision 5 —
Auth	Amazon Cognito User Pool	Admin authentication. JWT issuance for protected dashboard routes.
AI — Embeddings	Bedrock Titan Text Embeddings	1024-dimensional embeddings. Fully AWS-native. Used for both ingestion and search.
AI — Generation	Bedrock Claude Haiku	LLM for RAG response generation. Temperature 0.3 for factual consistency.
RAG Orchestration	AWS Lambda (Python)	Receives query, embeds, runs pgvector HNSW search, assembles context, and generates response.
PDF Ingestion	AWS Lambda (Python)	S3 event trigger. Chunks PDF, generates embeddings, writes to pgvector.
Relational + Vector DB	Amazon RDS PostgreSQL + pgvector	Small minimum for production. Leads, products, knowledge_chunks.
Email	Amazon SES	Referral and consultation confirmation emails plus internal staff alerts.
Secrets	AWS Secrets Manager	DB credentials, Stripe secret key, Stripe webhook secret, and all API keys.
Monitoring	Amazon CloudWatch	Lambda execution logs, RDS performance metrics, billing alarms.
DNS + CDN	Route 53 + CloudFront	Custom domain routing and global CDN for static assets.
Security	AWS WAF on API Gateway	Rate limiting: 20 chat requests/min/IP. 10 admin requests/min/IP.
Payments	Stripe	The single authorized third-party exception to the AWS-native principle.

5.2 System Data Flows

Flow 1: Form Submission (Referral or Consultation)

- User fills /refer or /contact form in the browser.
- Browser POSTs to Next.js API route (/api/refer or /api/consult) over HTTPS.
- API route validates the payload with Zod. Invalid input returns 400 immediately.
- Valid payload: API route calls pg client to INSERT into RDS leads table.
- API route invokes SES to send two emails: confirmation to submitter + alert to PHM team inbox.
- GA4 fires form_submit event (from the frontend, after receiving 200 from API).
- Browser renders inline success state. No page redirect.

Flow 2: Chatbot Message

- User types a message in the chat widget. Frontend POSTs to /api/chat.
- Next.js API route strips HTML, enforces 500-char limit, validates allow-list.
- API route invokes RAG Lambda with sanitized query + session history (last 5 turns).
- RAG Lambda calls Bedrock Titan to embed the query (1024-dim vector).
- Lambda runs cosine similarity against 5 cluster_centroids rows. Returns highest-similarity cluster_tag in under 1ms.

- Lambda runs HNSW search on knowledge_chunks WHERE cluster_tag = ORDER BY embedding <=> query_vec LIMIT 5.
- Lambda assembles context block from top-5 chunk contents, capped at 2000 tokens.
- Lambda calls Bedrock Claude Haiku with system prompt + context + conversation history + query. Streaming enabled.
- Streamed tokens are piped back through the Next.js API route to the browser. Chat widget renders tokens progressively.
- On completion, a CTA button is appended. If response mentions referral: Refer a Patient → /refer. Otherwise: Request a Consultation → /contact.

Flow 3: Knowledge Base Update

- PHM staff drops a new or updated PDF into the S3 knowledge-base bucket.
- S3 fires an ObjectCreated event to the ingestion Lambda.
- Lambda downloads the PDF, splits into 300–500 token chunks with 50-token overlap.
- Lambda calls Bedrock Titan for each chunk to generate a 1024-dim embedding.
- Lambda INSERTs all chunk rows into knowledge_chunks with the correct cluster_tag.
- Lambda recomputes all 5 cluster centroids as the mean of their respective chunk embeddings. UPSERTs into cluster_centroids.
- Chatbot reflects updated content within minutes. No redeployment, no developer involvement.

Flow 4: Admin Login and Lead Management

- PHM staff navigates to /admin/login. Next.js middleware checks for a Cognito JWT. None found: renders login form.
- Staff enters email and password. Amplify Auth SDK calls Cognito User Pool.
- Cognito verifies credentials and issues an ID token and access token. Stored in Zustand (memory only, never localStorage).
- Subsequent requests to /admin/* include the JWT in the Authorization header.
- Next.js middleware verifies the JWT by fetching Cognito's JWKS endpoint. Invalid or expired tokens redirect to /admin/login.
- Admin fetches lead data via /api/admin/leads. API route queries RDS leads table. Data rendered in dashboard.
- Admin updates lead status via PATCH /api/admin/leads/:id. Change logged to CloudWatch with actor identity and timestamp.

Flow 5: E-Commerce Payment Pipeline (New in Revision 5)

New in Revision 5. This flow describes the complete purchase lifecycle from customer checkout to confirmed database record.

- Customer browses /shop, selects a product, and clicks Add to Cart or Buy Now.
- Cart state is managed in Zustand (memory only). Customer proceeds to checkout.
- Frontend calls the Next.js API route to create a Stripe PaymentIntent server-side. The secret key is read from AWS Secrets Manager. Stripe returns a client_secret.
- The client_secret is passed to Stripe Elements on the frontend. The customer enters card details directly into Stripe-hosted iframe components. Card data never touches PHM servers — PCI scope is entirely on Stripe's side.
- Stripe processes the payment. On success, Stripe fires a charge.succeeded webhook event to the registered endpoint.
- Stripe's webhook hits /api/webhooks/stripe on the PHM backend.

- The API route reads the raw request body and calls `stripe.webhooks.constructEvent()` with the `STRIPE_WEBHOOK_SECRET` from AWS Secrets Manager. Any invalid signature returns 400 immediately — the event is not processed.
- On a valid `charge.succeeded` event, the API route writes a transaction record to PostgreSQL, including the Stripe PaymentIntent ID, amount, and product reference. The record is immutable — status updates arrive via subsequent Stripe webhook events, never via admin overrides.
- GA4 fires a purchase event with the transaction value and product details. No card data is included in the GA4 payload.
- An SES confirmation email is dispatched to the customer. An internal SES alert is sent to the PHM team inbox.

Flow 6: Product Image Upload (New in Revision 5)

- Admin navigates to `/admin/products` and clicks Add Product or Edit Product.
- Admin selects an image file. The frontend immediately calls `URL.createObjectURL(file)` to generate a local object URL and renders it as an inline preview. No network request is made at this stage.
- Admin confirms the upload. The frontend POSTs to `/api/upload/presigned` with the filename and MIME type. Cognito JWT is included in the Authorization header.
- The API route verifies the JWT, constructs a time-limited S3 presigned PUT URL scoped to the product images prefix in the S3 bucket, and returns it to the frontend along with the final public `image_url`.
- The frontend uses the presigned URL to PUT the binary image file directly to S3. The Next.js server never handles the binary data.
- On successful S3 upload, the frontend stores the returned `image_url` in the product form state. When the admin saves the product, `image_url` is included in the POST `/api/admin/products` payload and written to the products table.

5.3 IAM Permissions — Least Privilege

Component	IAM Permissions Granted
Ingestion Lambda	S3: <code>GetObject</code> on knowledge-base bucket only. RDS: <code>INSERT</code> on <code>knowledge_chunks</code> and <code>clus</code>
RAG Lambda	RDS: <code>SELECT</code> on <code>knowledge_chunks</code> and <code>cluster_centroids</code> . Bedrock: <code>InvokeModelWithResp</code>
Next.js API Routes (Amplify)	No direct database access. <code>Lambda:InvokeFunction</code> for RAG Lambda only. Cognito: <code>cognito-id</code>
SES Lambda / API Route	<code>SES:SendEmail</code> scoped to verified sender identity only.
Admin API Routes	RDS: <code>SELECT</code> and <code>UPDATE</code> on <code>leads</code> table. <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> on <code>products</code> table. N

RDS is in a private VPC subnet with no public endpoint. Only Lambda functions attached to the same VPC security group can connect. Amplify has zero direct database access. This is enforced at the network layer, not just IAM.

6. RAG Chatbot Implementation

Unchanged from Revision 4. The RAG chatbot architecture, behavioral contract, system prompt, knowledge base clusters, and guardrails are identical to Revision 4. The chatbot has no connection to the e-commerce layer and never references products or pricing. Full specification follows.

Chatbot Behavioral Contract

1. The chatbot **NEVER** collects any form data: no names, no phone numbers, no email addresses, no health information.
2. The chatbot **ONLY** guides users. It answers PHM questions, describes programs, and explains processes.
3. Every informational response ends with one of two CTAs: *Refer a Patient* (→ /refer) or *Request a Consultation* (→ /contact).
4. On any medical emergency keyword (chest pain, can't breathe, unconscious, call 911, etc.), the chatbot responds **ONLY** with "Please call 911 immediately." No other text.
5. On any referral intent, the chatbot says "Please use our referral form" and shows the CTA. It does not take down any information.
6. The chatbot never pretends to be a doctor, never gives diagnoses, and never contradicts a physician.

6.1 Why RAG, Not Fine-Tuning

Fine-tuning a custom model on PHM data costs tens of thousands of dollars, takes months, and becomes stale whenever PHM updates a program. RAG retrieves live content from PHM's documents at query time. Drop a new PDF in S3, and the chatbot reflects the change within minutes. No retraining. No additional cost. No developer involvement for content updates.

6.2 AWS-Native AI Stack

AI Function	AWS Service	Configuration
Chunk embedding (ingestion)	Amazon Bedrock Titan Text Embedding	1024 dimensions. Called once per chunk during PDF ingest
Query embedding (runtime)	Amazon Bedrock Titan Text Embedding	1024 dimensions. One call per user message.
Intent routing	pgvector cosine similarity — no LLM call	Cosine similarity of query embedding vs. 5 pre-computed ce
Response generation	Amazon Bedrock Claude Haiku	Temperature 0.3 for factual consistency. Max tokens 1024.

6.3 Full RAG Pipeline

Step	Component	Detail
1. User message	Frontend chat widget	User types query. Frontend POSTs to /api/chat with message and session ID
2. Sanitize & validate	Next.js API route	HTML stripped. Max 500 characters enforced. Allow-list pattern checked. Mal
3. Query embedding	Bedrock Titan v2	Single Bedrock API call. Returns 1024-dimensional float vector for the query
4. Intent routing	pgvector centroid similarity	Query vector compared against 5 cluster centroids using cosine similarity. Re

Step	Component	Detail
5. Vector search	RDS pgvector HNSW	Top-5 chunks by cosine similarity from knowledge_chunks WHERE cluster_t
6. Context assembly	RAG Lambda	Top-5 chunk content fields joined into a single context block. Hard cap at 200
7. Generation	Bedrock Claude Haiku	System prompt + context block + last 5 conversation turns + current user me
8. Response delivery	Frontend widget	Streamed tokens rendered progressively. On completion: CTA button append

6.4 System Prompt

The system prompt is the primary safety and compliance layer. It is injected on every Bedrock call server-side and is never exposed to the frontend or visible to users.

You are a virtual assistant for Priority Home Monitor (PHM), a remote patient monitoring service based in Waxahachie, TX.

RULES – follow these exactly, with no exceptions:

1. Answer ONLY using the provided context from PHM official documentation.
Do not invent services, pricing, or clinical claims.
2. If the context does not contain the answer, say exactly:
'I don't have that specific information. Please call us at +1 972-573-4015.'
3. If a user describes a medical emergency or any symptoms requiring immediate attention, respond ONLY with: 'Please call 911 immediately.'
Do not add any other text.
4. Never collect, ask for, or acknowledge personal health information, patient names, phone numbers, email addresses, or any other identifying data.
5. If a user wants to refer a patient or submit any information, direct them to the referral form: 'Please use our Refer a Patient form.'
6. Do not provide medical diagnoses, treatment advice, or medication guidance.
7. End every informational response with one CTA: 'Refer a Patient' or 'Request a Consultation'.

6.5 Knowledge Base Clusters

Cluster	Label	PDF Convention	Content Scope
A	General Identity	A_*.pdf	Mission, tagline, key benefits, daily monitoring overview, about PHM
B	Medical Programs	B_*.pdf	COPD, Heart Failure, Diabetes, Hypertension, OSA, Sleep (Adult/Pediatric)
C	Safety & Fall Detection	C_*.pdf	Fall statistics, 24/7 passive monitoring, false alert rate (<1/month), wearable
D	Clinical Team	D_*.pdf	Medical Director, NPs, PAs, CMAs, virtual staff, care coordination model
E	Logistics & Conversion	E_*.pdf	Contact info, location (Waxahachie TX), referral process, consultation flow,

7. Admin Panel

7.1 Purpose

The admin panel is a protected internal dashboard accessible only to authenticated PHM staff. Revision 5 expands its scope to include two primary interfaces: the leads management view (unchanged from Revision 4) and the new Product Catalog CRUD interface for managing the /shop inventory. The chatbot has no connection to the admin panel. No clinical data is stored or accessible here.

7.2 Admin Routes

Route	Access	Description
/admin/login	Public (unlisted)	Cognito sign-in form. No link to this page exists on the public site.
/admin/dashboard	admin + superadmin	Summary stats: total submissions, new referrals count, status breakdown
/admin/leads	admin + superadmin	Paginated, filterable table of all form submissions. Filter by type, status, d
/admin/leads/[id]	admin + superadmin	Detail view for a single submission. Status update dropdown. Internal not
/admin/products	admin + superadmin	Product catalog CRUD interface. Add, edit, and archive products. Im
/admin/users	superadmin only	Create and deactivate admin user accounts. Assign roles. Returns 403 fo

7.3 Authentication — Amazon Cognito

Cognito User Pool handles all admin authentication. No custom auth logic and no passwords in the database. Cognito issues signed JWTs that are verified server-side on every protected API route using the Cognito JWKS endpoint.

Step	Implementation
Admin login	Staff navigates to /admin/login. React form calls Amplify Auth.signIn(email, password).
JWT issuance	Cognito issues an ID token (identity claims) and access token (API authorization). Stored in Zustand
Route protection	Next.js middleware (src/middleware.ts) intercepts all /admin/* requests. Checks Authorization heade
Token verification	API routes fetch Cognito JWKS (cached for 1 hour) and verify the JWT signature and expiry before p
Token refresh	Amplify Auth SDK handles silent refresh using the Cognito refresh token. Access tokens expire in 1
Logout	Cognito global sign-out revokes all tokens for the user. In-memory tokens cleared. Redirect to /admi
Brute-force protection	Cognito has built-in account lockout after 5 failed login attempts. reCAPTCHA v3 on the login form a

7.4 Role-Based Access Control (RBAC)

Role	Permissions
admin	View all leads. Update lead status. View and manage product catalog (add, edit, archive). Cannot delete leads

Role	Permissions
superadmin	All admin permissions. Create and deactivate admin accounts. Export leads as CSV. Access to all routes incl

7.5 Cognito + Backend API: Separation of Concerns

This subsection clarifies the strict separation between Amazon Cognito (Identity Provider) and the Next.js backend (Authorization Enforcer). Understanding this boundary is critical for correctly implementing and auditing the RBAC system.

Layer	Responsibility	What It Does NOT Do
Amazon Cognito (Identity Provider)	Issues signed JWTs that contain the user's identity claims. NOT defining the business logic or access control	Cognito does NOT define the business logic or access control
Next.js Middleware (Route Guard)	Intercepts every /admin/* request. Verifies the JWT signature and checks for the required role	Does not sign JWTs or check for the required role
Backend API Helper Functions (Authorization Enforcer)	Every protected API route calls a hasPermission() helper. The helper checks the role against the required role	The helper does not sign JWTs or check for the required role

Key Rule: Cognito is the identity layer. The Next.js backend is the authorization layer. Never implement business access control in Cognito Groups or Cognito Triggers. All permission logic lives in the API route hasPermission() helper, making it auditable in a single location.

7.6 Security Rules

- Admin routes are not linked from anywhere on the public site. No search engine will index /admin/*.
- All /admin/* routes require a valid Cognito JWT. The middleware check happens before any DB query runs.
- Admin panel has its own WAF rule: 10 requests per 5 minutes per IP on /admin/*.
- All admin actions (status updates, product edits, archiving) are logged to CloudWatch with actor email and timestamp.
- RDS lead and product data is never sent directly to the browser. All access goes through verified API routes.
- JWT tokens stored in memory only. If the browser tab is closed, the admin must log in again. This prevents token theft via XSS in localStorage.
- Product DELETE operations are prohibited at the application layer. The PATCH endpoint accepts stock_status = "archived" only. No DELETE SQL statement is ever issued against the products table.

8. Analytics — Google Analytics 4

8.1 Setup

GA4 is installed via the Next.js Script component in the root layout (app/layout.tsx) with strategy='afterInteractive'. This prevents the analytics script from blocking page render and maintains Lighthouse performance scores.

```
// app/layout.tsx
import Script from 'next/script'

const GA_ID = process.env.NEXT_PUBLIC_GA_MEASUREMENT_ID

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        {children}
        <Script
          src={`https://www.googletagmanager.com/gtag/js?id=${GA_ID}`}
          strategy='afterInteractive'
        />
        <Script id='ga4-init' strategy='afterInteractive'>
          {`window.dataLayer=window.dataLayer||[];
          function gtag(){dataLayer.push(arguments);}
          gtag('js', new Date());
          gtag('config', '${GA_ID}', { anonymize_ip: true });`}
        </Script>
      </body>
    </html>
  )
}
```

8.2 Custom Event Strategy

Event Name	When to Fire	Parameters to Include
cta_click	User clicks any primary or secondary CTA (Refer a Patient)	cta_location (page path)
program_view	User lands on any /programs/* page	program_name ('COPD'), source_page
form_start	User focuses the first field on /refer-to-us, /create-account, /contact	form_type ('refer', 'contact')
form_submit	Successful API response (200) from any type of program submission	form_type, program_name, source_page
form_error	Zod validation fails on form submit	form_type, error_field
chat_open	User clicks the chatbot floating bubble	page_path

Event Name	When to Fire	Parameters to Include
chat_message_sent	User submits a chat message (no message content — privacy)	page_path
chat_cta_click	User clicks the CTA appended to chatbot response	cta_label, response_cluster
emergency_trigger	System prompt 911 fallback fires in page path (no message content ever)	page_path
shop_product_view	User views a product detail on /shop	product_id, product_name, product_type
add_to_cart	User clicks Add to Cart on /shop	product_id, product_name, price
purchase	Stripe payment confirmed (fired after successful Stripe PaymentIntent ID, via Stripe Element, product ID)	transaction_id (Shop PaymentIntent ID), wallet_id, stripe_token, product_id
cookUnity_click	User clicks the external CookUnity link	source_page

Pitfall: Always fire form_submit and purchase events after a successful API response, not on button click. Firing on click counts failed attempts as conversions and corrupts data.

8.3 Conversion Events

Configure the following as conversions in the GA4 admin console (Admin > Events > Mark as conversion):

- form_submit where form_type = 'referral' — primary healthcare conversion
- form_submit where form_type = 'consultation' — secondary healthcare conversion
- cta_click where cta_label = 'Request a Platform Demo' — B2B provider conversion
- purchase — e-commerce conversion (new in Revision 5)

8.4 Privacy Configuration

- IP anonymization enabled in the gtag config (anonymize_ip: true). Default in GA4 but set explicitly.
- No patient health information, chat message content, or identifying data is sent to GA4 at any point.
- Event parameters are limited to page paths, form types, CTA labels, product IDs, and transaction values.
- Card numbers, billing addresses, and Stripe tokens are never included in any GA4 event.
- This keeps the analytics implementation outside HIPAA scope by design.

9. Security & HIPAA-Ready Structure

9.1 Current HIPAA Position

PHI Boundary — Read This Before Adding Any New Data Fields

This platform is a public-facing marketing, lead-generation, and e-commerce site. It does NOT store, process, or transmit Protected Health Information (PHI) as defined by HIPAA. Referral forms collect contact information only. Product transactions collect payment information handled entirely by Stripe. No diagnosis, medication, insurance, lab results, or clinical records are collected anywhere. This is a deliberate architectural constraint. Any addition of clinical data fields requires a HIPAA review first.

9.2 Security Measures

Threat	Mitigation
SQL injection	Parameterized queries via the pg library throughout. No raw SQL string concatenation at any endpoint.
Prompt injection	Chat input stripped of HTML, capped at 500 chars, validated against an allow-list before any Bedrock invocation.
Credential exposure	All secrets — including Stripe secret key and webhook secret — in AWS Secrets Manager. No .env files.
Unauthorized database access	RDS in private VPC subnet, no public endpoint. Only Lambda functions in the same VPC security group can connect.
Admin route access	Cognito JWT required on all /admin/* routes. Server-side JWT verification on every request before processing.
Admin brute-force	Cognito locks account after 5 failed attempts. reCAPTCHA v3 on login form. WAF: 10 requests per IP per minute.
Chat abuse and DDoS	AWS WAF: 20 chat requests per minute per IP on /api/chat. API Gateway throttling as a secondary defense.
Form spam	reCAPTCHA v3 (invisible) on /refer, /consult, and /contact. Server-side token verification before form processing.
Data in transit	HTTPS enforced site-wide via CloudFront. HTTP auto-redirects to HTTPS at the CDN edge.
Data at rest	RDS encryption via AWS KMS. S3 server-side encryption (SSE-S3) on all buckets including the product catalog.
JWT theft via XSS	JWTs stored in Zustand memory only. Not in localStorage or cookies. Tab close clears all tokens.
Bedrock conversation logging	Invocation logging disabled in production. Conversation content never enters CloudWatch or any logs.
Stripe webhook spoofing	All /api/webhooks/stripe requests verified via stripe.webhooks.constructEvent() with the webhook secret.
PCI DSS scope	Card data never touches PHM servers. Stripe Elements renders card inputs in a Stripe-hosted iframe.
Product hard deletes	DELETE statements against the products table are prohibited at the application layer. Archiving via soft deletes.

10. Cost Optimization

10.1 Monthly Cost Estimate at Launch

Service	Configuration	Est. Monthly Cost
AWS Amplify Gen 2	Starter plan. CI/CD from GitHub included.	~\$0–15
Amazon RDS (db.t3.small)	20GB SSD, Single-AZ, 7-day backup retention.	~\$25–35
AWS Lambda	< 1 million invocations per month.	~\$0 (free tier)
Bedrock Titan Embeddings v2	~1M tokens/month (ingestion + query-time embeddings).	~\$1–2
Bedrock Claude Haiku	~500K input tokens per month.	~\$5–15
Amazon S3	5GB storage + request charges (PDFs + product images).	~\$2–3
Amazon Cognito	Up to 50,000 MAU free tier. PHM is far below this.	~\$0
Amazon SES	~1,000 emails per month.	~\$0.10
Amazon CloudFront	~10GB transfer per month.	~\$1
AWS WAF	1 Web ACL + rate-based rules.	~\$5–10
Amazon Route 53	Hosted zone + DNS queries.	~\$1–2
Amazon CloudWatch	Log storage + alarms.	~\$2–5
AWS Secrets Manager	~4 secrets stored (added Stripe keys).	~\$2
Stripe	Per-transaction fee of 2.9% + 30¢. No fixed monthly fee.	~\$0 fixed + variable
Total (excluding Stripe transactions)		~\$44–90/month

Stripe Cost Model: Stripe introduces \$0.00 in fixed daily or monthly infrastructure costs. It operates entirely on a per-transaction basis at 2.9% + 30¢ per successful charge. There are no subscription fees, no monthly minimums, and no API call charges. This means the serverless baseline run rate for the PHM platform remains completely unaffected on days with zero transactions. Stripe costs scale linearly and exclusively with revenue generated.

10.2 Key Optimization Decisions

Serverless-First (Already Implemented)

- Lambda runs only when invoked. At low traffic, Lambda cost is essentially zero. No idle compute charges.
- Next.js on Amplify is edge-deployed. Static pages serve from CloudFront with no backend involvement.
- PDF ingestion Lambda runs only when a new document is uploaded. Not a recurring cost.

Single RDS Instance for All Storage

- pgvector on RDS eliminates Pinecone (~\$70/month) or Weaviate (~\$50+/month). The same RDS instance handles leads, products, vectors, and admin users.
- Use Single-AZ at launch. Multi-AZ adds ~\$28/month in redundancy. Add it only when uptime requirements justify it.

Bedrock Cost Controls

- Claude Haiku is the cheapest Bedrock generation model. Do not upgrade to Sonnet unless quality is demonstrably insufficient.
- Hard cap the RAG context at 2000 tokens per call. Prevents runaway token costs on verbose PDFs.
- Consider caching frequent chatbot query embeddings with ElastiCache at 10K+ daily queries. Not needed at launch.
- Set a CloudWatch billing alarm at \$80/month. If triggered, investigate unusual Lambda or Bedrock invocation patterns.

S3 Image Delivery

- All product images must be served through CloudFront, not directly from S3. CloudFront egress: ~\$0.0085/GB. S3 egress: ~\$0.09/GB. 10x cheaper.
- Enable S3 Intelligent-Tiering on the knowledge-base and product-images buckets. Infrequently accessed objects shift to cheaper storage tiers automatically.

10.3 Cost by Traffic Tier

Traffic Level	Users/Day	Changes Needed	Est. Monthly Total
Launch	< 100	None	~\$44–90
Early growth	100–500	None — Lambda and Amplify scale automatically	\$60–115
Active growth	500–2,000	Add RDS Proxy (~\$15/month). Upgrade to db.t3.medium if needed.	\$90–155
Scale	2,000–10,000	RDS read replica, consider ElastiCache for embeddings	\$160–265
High volume	10,000+	RDS Multi-AZ, Lambda Reserved Concurrency, S3 Intelligent-Tiering	\$300–510

11. Scalability Architecture

11.1 Frontend — CloudFront CDN

All 16 program pages, the home page, the /shop storefront, and all static content are rendered at build time (Next.js Static Site Generation) and deployed to AWS CloudFront's 400+ global edge locations. These pages handle millions of requests with zero backend involvement and near-instant load times.

Page Type	Rendering Strategy	Examples
SSG (build-time render)	CloudFront edge — no origin hit	/, /programs, /programs/*, /contact, /refer, /platform, /shop, /admin
Server-rendered	Amplify edge — requires live data	/admin/* dashboard pages, /admin/products (product list requires

11.2 Lambda Concurrency

Setting	Value	Reason
Reserved Concurrency (RAG Lambda)	200	Prevents a traffic spike from consuming the entire regional concurrency pool.
Provisioned Concurrency (RAG Lambda)	5	Eliminates cold-start latency for the first 5 concurrent users. Cost: ~\$3–5/month.
Reserved Concurrency (Ingestion Lambda)	5	Low value is fine. Ingestion runs infrequently and cold starts are acceptable.
Timeout (RAG Lambda)	30 seconds	Bedrock streaming can take up to 10 seconds. Set timeout with headroom.
Memory (RAG Lambda)	512MB	psycopg2 + numpy for vector ops. 256MB is too tight.

11.3 Database Scaling

Scenario	Solution	Cost Impact
Admin dashboard slow due to analytics queries	Add RDS read replica (db.r5.xlarge) to primary. Route analytics queries from admin panel to the replica.	~\$28/month
Lambda exhausting PostgreSQL connections	Add RDS Proxy. Offloads connection management to proxy. Prevents "too many connections" errors.	~\$15/month
Vector search slowing as knowledge_chunks grows	Increase HNSW fast_5k_shards to 128. Consider partitioning knowledge_chunks by cluster_tag.	~\$15/month
Product catalog growing to thousands of SKUs	Add B-tree index on products(stock_status, created_at DESC). Covers the most common /shop	~\$15/month
RDS storage filling up	RDS auto-scales storage by default. No action required.	~\$15/month

11.4 Scaling Upgrade Order

Do these in order — only when traffic justifies it.

- Step 1 (at launch): Set Provisioned Concurrency = 5 on RAG Lambda. ~\$3–5/month. Immediate UX improvement for first users.
- Step 2 (at 500+ daily users): Add RDS Proxy. ~\$15/month. Prevents Lambda connection exhaustion.
- Step 3 (at 1,000+ daily users): Add RDS read replica. ~\$28/month. Offloads admin and analytics traffic from primary.

- Step 4 (at 5,000+ daily users): Add ElastiCache (Redis) for embedding query cache. ~\$15/month. Reduces Bedrock calls for repeated questions.
- Step 5 (at 10,000+ daily users): RDS Multi-AZ. ~\$28/month added. Provides automatic failover for high availability.

12. Implementation Roadmap

This is the step-by-step execution guide. Build order: website first (including /shop), chatbot second, AWS deployment third. This order eliminates wasted cloud spend on broken builds and lets you validate the entire website and e-commerce flow before adding AI complexity.

Before You Start — Prerequisites (Complete All of These First)

- Node.js 20+ installed. Verify: `node -v`
- Python 3.11+ installed. Verify: `python3 --version`
- AWS CLI installed and configured with your access key: `aws configure`
- AWS SAM CLI installed: `pip install aws-sam-cli` — verify: `sam --version`
- Git and GitHub account. Create a new private repository for this project.
- AWS account created. Set billing alerts at \$50 and \$100 in the Billing console.
- Google Analytics 4 property created at `analytics.google.com`. Note your Measurement ID (G-XXXXXXXXXX).
- Amazon SES: verify both `priorityhomemonitor@gmail.com` and any domain email as sender identities.
- Docker Desktop installed (for running local PostgreSQL during development).
- Stripe account created at `stripe.com`. Note your publishable key and secret key. Set up a webhook endpoint and note the webhook signing secret.

Phase 1 — Local Setup & AWS Prerequisites (Week 1)

Goal: A running Next.js app locally, a local database ready for development, and the AWS services that need to exist before you write any application code (RDS and Cognito). You will not deploy the website yet.

Step 1: Create the Next.js Project

- Open your terminal in the folder where you want the project.
- Run: `npx create-next-app@latest phm-platform --typescript --tailwind --eslint --app --src-dir`
- Answer the prompts: TypeScript: Yes. Tailwind: Yes. ESLint: Yes. App Router: Yes. src/ directory: Yes. Import alias: `@/*`.
- `cd phm-platform`
- Install core dependencies: `npm install zustand react-hook-form @hookform/resolvers zod lucide-react axios pg aws-amplify @aws-amplify/auth react-google-recaptcha-v3`
- Install Stripe dependencies: `npm install stripe @stripe/stripe-js @stripe/react-stripe-js`
- Install Shadcn/UI: `npx shadcn-ui@latest init` — choose Default style, Slate base color, yes to CSS variables.
- Install common Shadcn components: `npx shadcn-ui@latest add button card input label dialog select textarea badge table`
- Verify the app runs: `npm run dev` — open `http://localhost:3000`.

Pitfall: Do NOT install `@types/gtag.js` — it conflicts with Next.js 14 type definitions. Declare `gtag` as `window.gtag` in a `src/types/global.d.ts` file instead.

Step 2: Configure Local PostgreSQL with Docker

- Start a local Postgres container: `docker run --name phm-local-db -e POSTGRES_PASSWORD=localpass -e POSTGRES_DB=phm -p 5432:5432 -d postgres:16`
- Connect to it: `psql -h localhost -U postgres -d phm -W` (password: localpass)
- Run the full schema SQL from Section 4.4 to create all tables (including products), install pgvector, and create all ENUMs.

- Create a `.env.local` file in the project root with `DATABASE_URL`, `NEXT_PUBLIC_GA_MEASUREMENT_ID`, `RECAPTCHA` keys, `COGNITO` IDs, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`, `STRIPE_SECRET_KEY`, and `STRIPE_WEBHOOK_SECRET`.
- Add `.env.local` to `.gitignore`. Verify: `git status` — `.env.local` should not appear.

Pitfall: Never commit `.env.local` to Git. This file contains your Stripe secret key and webhook secret. If credentials are pushed to GitHub, rotate them immediately in both AWS Secrets Manager and the Stripe dashboard.

Steps 3–4: Provision RDS and Cognito

Follow Steps 3 and 4 from the Revision 4 roadmap exactly. No changes required. Ensure the products table and new ENUMs from Section 4.4 are applied to the production RDS instance when you run the schema setup in Step 3.

Phase 2 — Build the Full Website Locally (Weeks 1–3)

Goal: All 16 program pages plus `/shop` built and navigable locally, all forms working end-to-end, the `/admin/products` panel functional with full CRUD and image preview, and the Stripe checkout flow validated in test mode. Everything confirmed locally before any deployment.

Steps 5–8: Build Design System, 16 Pages, Forms, Admin Panel

Follow Steps 5 through 8 from the Revision 4 roadmap exactly. No changes required for these steps. The existing program pages, forms, and leads admin panel are built as specified in Revision 4.

Step 8a: Build the `/shop` Storefront

- Create `src/app/shop/page.tsx`. This is a server component that fetches products from the database on the server side before rendering. Query: `SELECT * FROM products WHERE stock_status != 'archived' ORDER BY created_at DESC`.
- Build the `ProductCard` component: product image (from `image_url` via CloudFront), name, price, type badge (Individual / Bundle), Add to Cart button.
- Implement cart state using Zustand. Cart persists in memory only — never in `localStorage`. Store: `{ items: CartItem[], addItem, removeItem, clearCart }`.
- Build the checkout flow: cart summary page → Stripe Elements card form → submit.
- Wire the `PaymentIntent` creation: on checkout, POST to a new `/api/stripe/create-intent` route. This route reads the `STRIPE_SECRET_KEY` from environment, calls `stripe.paymentIntents.create({ amount, currency: 'usd' })`, and returns the `client_secret`.
- Pass `client_secret` to Stripe Elements' Elements provider component. Render `CardElement` inside a form. On submit, call `stripe.confirmCardPayment(clientSecret, { payment_method: { card: cardElement } })`.
- On Stripe Elements `onSuccess` callback, fire GA4 purchase event and redirect to a `/shop/confirmation` page.
- Test using Stripe test card 4242 4242 4242 4242, expiry any future date, CVC any 3 digits. Verify the payment appears in the Stripe test dashboard.

Step 8b: Build the `/admin/products` Panel

- Create `src/app/admin/products/page.tsx`. Protected by the existing Cognito JWT middleware.
- Build the product table: name, price, type, `stock_status`, image thumbnail, Edit and Archive buttons. No Delete button.
- Build the Add / Edit Product form: name (text), description (textarea), price (number, min 0.01), type (dropdown: individual / bundle), `stock_status` (dropdown: in_stock / out_of_stock / archived), image upload field.

- Implement image preview: in the image upload onChange handler, call `const previewUrl = URL.createObjectURL(e.target.files[0])` and set it as the src of a preview img element. This renders the image instantly without any network request.
- On form submit with a new image file: first POST to `/api/upload/presigned` with `{ filename, contentType }`. The API route verifies the JWT, generates an S3 presigned PUT URL, and returns `{ presignedUrl, imageUrl }`.
- Use the presignedUrl to PUT the file directly to S3 from the frontend: `await fetch(presignedUrl, { method: 'PUT', body: file, headers: { 'Content-Type': file.type } })`.
- Include the returned imageUrl in the product form payload when calling POST or PATCH `/api/admin/products`.
- Wire PATCH `/api/admin/products/:id` for the Archive action. Send `{ stock_status: 'archived' }`. No delete endpoint exists.
- Test: add a product with an image, verify it appears on `/shop`. Archive the product, verify it disappears from `/shop` but remains visible in `/admin/products`.

Step 9: Configure AWS Webhook for Stripe

- In the Stripe dashboard: Developers → Webhooks → Add endpoint. Enter `https://your-domain.com/api/webhooks/stripe`.
- Select the events to listen for: `charge.succeeded`, `payment_intent.payment_failed`, `payment_intent.canceled`.
- Note the Signing Secret. Store it in AWS Secrets Manager as `STRIPE_WEBHOOK_SECRET`.
- Build the `/api/webhooks/stripe` route. This route must bypass the standard Next.js body parser. In Next.js App Router, read the raw body using: `const buf = await request.arrayBuffer(); const rawBody = Buffer.from(buf);`
- Call `stripe.webhooks.constructEvent(rawBody, request.headers.get('stripe-signature'), webhookSecret)`. Wrap in try-catch. On error, return 400.
- On a `charge.succeeded` event, INSERT a transaction record into PostgreSQL with: `stripe_payment_intent_id`, `amount`, `currency`, `status`, product reference, customer email (from Stripe metadata), `created_at`.
- Test locally using the Stripe CLI: `stripe listen --forward-to localhost:3000/api/webhooks/stripe`. Trigger a test event: `stripe trigger charge.succeeded`.

Pitfall: The `/api/webhooks/stripe` route MUST receive the raw unparsed request body for Stripe signature verification. If Next.js parses the body before this route runs, `constructEvent()` will fail on every request. Use `request.arrayBuffer()` directly.

Phase 3 — Build the AI Chatbot (Weeks 3–4)

Goal: PDF ingestion pipeline working, RAG Lambda producing correct answers, chatbot widget live on all pages (including `/shop`) following the behavioral contract. The chatbot does not reference products or pricing from the shop catalog. Follow Steps 9 through 12 from the Revision 4 roadmap exactly.

Phase 4 — Deploy to AWS (Weeks 4–5)

Goal: The entire platform running on AWS production infrastructure. Follow Steps 13 through 17 from the Revision 4 roadmap with the following additions:

- In Amplify environment variables, add `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` (safe to expose publicly) and confirm `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` are in AWS Secrets Manager, not in Amplify env vars.
- Switch Stripe from test mode to live mode: replace test API keys with live keys in Secrets Manager. Update the Stripe webhook endpoint URL to the production domain.
- Verify the Stripe webhook is receiving events in the Stripe live dashboard after a real test purchase.
- Configure S3 CORS on the product-images bucket to allow PUT requests from your production domain.

Phase 5 — QA & Production Launch (Weeks 5–6)

Follow Steps 18 through 21 from the Revision 4 roadmap with the following additional QA items:

- End-to-end checkout test in production: complete a real purchase with a \$0.01 test product, verify the Stripe webhook fires, verify the transaction record appears in PostgreSQL, verify the confirmation email arrives.
- Test the No Hard Delete policy: attempt to delete a product from the admin panel. The Delete button should not exist. Confirm only the Archive action is available.
- Verify URL.createObjectURL image preview: upload a product image in /admin/products and confirm the preview renders before the upload to S3 completes.
- Verify /shop filters archived products: archive a product in /admin/products, confirm it no longer appears on /shop without a page reload.
- GA4 validation: confirm purchase event appears in GA4 DebugView with correct transaction_id and value after a test checkout.

Phase Checklist — Final Go-Live

- ✓ All 16 program pages + /shop render correctly on production with final approved copy
- ✓ Stripe is in live mode. Test purchase confirms charge in Stripe live dashboard.
- ✓ Stripe webhook fires in production and transaction record appears in PostgreSQL.
- ✓ No Hard Delete enforced: no Delete button visible in /admin/products UI.
- ✓ URL.createObjectURL preview renders before S3 upload completes.
- ✓ Product archiving hides items from /shop immediately on next page load.
- ✓ Lighthouse Performance > 90 on /, /shop, and at least two program pages
- ✓ Lighthouse Accessibility > 95 on all tested pages
- ✓ All 6 emergency keyword tests return only the 911 message
- ✓ Chatbot never collects data on referral intent
- ✓ SES is out of sandbox — test emails arrive at real addresses
- ✓ GA4 conversions configured including the purchase event
- ✓ CloudWatch billing alarm set at \$80/month
- ✓ Bedrock model invocation logging is disabled
- ✓ Stakeholder sign-off obtained

13. Open Questions for Stakeholder Sign-Off

The following decisions require explicit direction. Development proceeds with the default assumption shown unless directed otherwise. All must be resolved before the phase indicated.

Question	Default Assumption	Resolve Before
Should the chatbot collect referral details?	Always chatbot collects details to never collect details.	Phase 3
Should the chatbot ever surface product recommendations?	Chatbot is subject to clinical. No product recommendations, no pricing, no shop links.	Phase 3
Is there a CookUnity affiliate or referral code appended to the CookUnity URL?	Not code appended to the CookUnity URL.	Phase 5
What is the hard content freeze date for all content copy used through Phase 4?	Planned for copy used through Phase 4.	Phase 5
Analytics: GA4 (current) or a privacy-focused alternative (e.g., Plausible, Fathom)?	GA4 as the primary analytics platform.	Phase 1 (if changing)
Is a patient portal or clinical dashboard needed for the upgrade?	No. See with 12 charts for the upgrade.	Before any BWA development is enabled now.
How many staff need admin panel access?	Two. Which roles? superadmin (owner) and admin (staff).	Phase 2
Has the custom domain been purchased?	Domain is not yet available for DNS setup.	Phase 4
Should admin users be able to delete inactive leads?	Not delete. Status change to "closed".	Phase 2
Who is the authorized product manager?	ED. The ED is the only one who can archive products.	Phase 4
Should the /shop support discount codes?	No promotional priority. Shop CTR is a post-launch without schema change.	Phase 4
Should order history be accessible to customers via a portal?	Customer only. Before Phase 5 only to admin staff via a future /admin endpoint.	Phase 5

Highlighted questions (rows 10–12) are new in Revision 5 and pertain specifically to the e-commerce layer. All other questions carry over from Revision 4 with their default assumptions unchanged.